Sobre todas las funcionalidades de Delta, realiza los siguientes ejercicios. Junto con el código, es necesario entregar un PDF breve que recoja las decisiones tomadas en cada ejercicio: qué función se ha usado, por qué, y cómo se interpreta el resultado.

EJERCICIO 7

Buscar información de Yahoo Finance que pueda servir para relacionar el sentimiento del experto con los datos de bolsa. Entonces, para cada empresa cogeremos su nombre, sector e industria y lo guardaremos todo en la capa broze.

Usamos las funciones:

- Descargar_informacion_empresas() de DescargaYahoo.py: obtiene el nombre, sector e industria de cada empresa
- Guarda_delta() de AlmacenamienntoDatos.py: guarda estos datos en data/lake/bronze/empresas

Usé sector porque los datos de sentimiento están organizados en sectores, guardo industria que puede ser útil. Quedarán separados de los precios porque describen a la empresa y no cambian cada día.

Podremos saber a que sector pertenece cada empresa y la relación de sus precios con el sentimiento, podremos estudiar si el sentimiento de un sector está relacionado con cambios en el precio.

EJERCICIO 8

Queremos limpiar la base de datos de sentimiento y guardarlos en la capa silver, para ello vamos a leer los datos desde bronze y se limpian y normalizan para guardarlos en silver

Usamos las funciones:

- leer_delta() y guardar_delta() de AlmacenamientoDatos.py: leen y guardan las tablas delta
- pasar_a_formato_largo() de LimpiezaDatos.py:transforma los datos si es necesario
- normalizar_sector() de LimpiezaDatos.py: unifica nombres de sectores
- limpiar_sentimiento() de LimpiezaDatos.py: realiza las principales operaciones de limpieza

El formato largo no es necesario en este caso porque el CSV ya lo tiene, pero es necesaria la generalización y normalizamos los sectores.

La tabla silver tendrá los mismos datos de sentimiento, pero ya limpios, con los tipos correctos y sin duplicados. Al comparar el numero de filas vemos si se han eliminado durante la limpieza.

EJERCICIO 9

He usado las funciones:

- limpiar_ibex() de LimpiezaDatos.py: limpia los datos de precios
- limpiar_empresas() y normalizar_sector() de LimpiezaDatos.py: limpian la información de las empresas
- leer_delta() y guardar_delta() de AlmacenamientoDatos.py: permiten leer y guardar las tablas Delta

Utilizo Ticker y Date para detectar duplicados porque cada empresa solo puede tener una cotización por día. La información de la empresa la guardamos aparte porque datos como el sector o la industria no cambian cada día y así no se repiten en todas las filas de precios.

Finalmente obtenemos dos tablas silver, una con las cotizaciones diarias y otra con la información de cada empresa. Al mostrar la quinta fila vemos si se ha realizado la limpieza.

EJERCICIO 10

Utilizamos las funciones:

- anadir_sector_a_precios() e integrar_precios_sentimiento() de IntegracionDatos.py: juntan los precios con la información de empresa y sentimiento
- construir_dimension_empresa() y construir_dimension_fecha() de IntegracionDatos.py: crean las dimensiones
- guardar_delta() de AlmacenamientoDatos.py: guarda las tablas de formato delta

Utilizamos el modelo estrella porque nos permite separar la información principal de las dimensiones. La tabla de hechos guarda las cotizaciones y el sentimiento, mientras que la empresa y la fecha se guardan aparte para no repetir datos innecesariamente.

La capa gold nos deja los datos preparados para el análisis, haciendo que sea mas sencillo realizar comparaciones o buscar relaciones.

EJERCICIO 11

Utilizamos las funciones:

- anadir_sector_a_precios() de IntegracionDatos.py: añade el sector a cada cotización
- anti_join() de IntegracionDatos.py: busca los registros que no tienen correspondencia
- leer_delta() de AlmacenamientoDatos.py: lee las tablas

Usamos la misma función anti_join() en ambos casos, ya que es la misma operación.

Las filas sin pareja aparecen porque las dos fuentes no siempre coinciden en las mismas fechas y sectores. Por eso, en la tabla de hechos solo se guardan los datos que sí tienen correspondencia.

EJERCICIO 12

Usamos las funciones:

- anadir_variacion_diaria() de Metricas.py: calcula la variación del precio respecto al día anterior
- anadir_rango_diario() de Metricas.py: calcula la diferencia entre el máximo y el mínimo del día.
- leer_delta() y guardar_delta() de AlmacenamientoDatos.py: lee y guarda la tabla delta

He elegido estas dos métricas porque son fáciles de interpretar y pueden ser útiles en un dashboard. La variación diaria permite ver si una acción ha subido o bajado, y el rango diario muestra cuánto se ha movido el precio durante ese día.

Al interpretar los resultados, si la variación diaria es positiva, el precio ha subido respecto al día anterior, y si es negativa, ha bajado. Un rango diario alto indica que hubo más diferencia entre el precio máximo y mínimo de ese día.

EJERCICIO 13

He usado las funciones:

- leer_delta() de AlmacenamientoDatos.py: lee las tablas de la capa gold
- exportar_parquet() de AlamacenamientoDatos.py: exporta las tablas a formato parquet

He mantenido el modelo en estrella, separando la tabla de hechos de las dimensiones de empresa y fecha. La tabla de hechos se particiona por Ticker para organizar mejor los datos de cada empresa y facilitar su lectura.

El resultado final queda preparado para Power BI, con una tabla de hechos y dos dimensiones relacionadas. Esto permite analizar los datos por empresa y fecha de una forma más organizada y eficiente.